

Chapter – 7

Advanced Topics in Java

Why Object Relational Mapping (ORM)?

When we work with an object – oriented system, there is a mismatch between the object model and the relational database. RDBMS represent data in a tabular format whereas object – oriented language, such as java or C# represent it as an interconnected graph of objects.

Consider the following java class with proper constructors and associated public function:

```
public class Employee
{
    private int id;
    private String first_name;
    private String last_name;
    private int salary;

    public Employee() {}
    public Employee(String fname, String lname, int salary)
    {
        this.first_name = fname;
        this.last_name = lname;
        this.salary = salary;
    }

    public int getId()
    {
        return id;
    }

    public String getFirstName()
    {
        return first_name;
    }

    public String getLastName()
    {
        return last_name;
    }

    public int getSalary()
    {
        return salary;
    }
}
```

Consider the above objects are to be stored and retrieved into the following RDBMS table

```
create table EMPLOYEE (  
    id INT NOT NULL auto_increment,  
    first_name VARCHAR(20) default NULL,  
    last_name VARCHAR(20) default NULL,  
    salary INT default NULL,  
    PRIMARY KEY (id)  
);
```

First problem, what if we need to modify the design of our database after having developed a few pages or our application? Second, loading and storing objects in a relational database exposes us to the following mismatch problem:

➤ **Granularity**

Sometimes you will have an object model, which has more classes than the number of corresponding tables in the database.

➤ **Inheritance**

RDBMSs do not define anything similar to inheritance, which is a natural paradigm in object – oriented programming languages.

➤ **Identity**

An RDBMS defines exactly one notion of ‘sameness’: the primary key. Java, however, defines both object identity (`a==b`) and object equality (`a.equals(b)`).

➤ **Association**

Object – oriented languages represent associations using object references where an RDBMS represents an association as a foreign key column.

➤ **Navigation**

The ways you access objects in java and in RDBMS are fundamentally different.

The Object – Relational Mapping (ORM) is the solution to handle all the above impedance mismatch.

What is ORM?

ORM stands for Object – Relational Mapping (ORM) is a programming technique for converting data between relational databases and object oriented programming languages such as java, C# etc.

Advantages of ORM over JDBC

- Lets business code access objects rather than DB tables.
- Hides details of SQL queries from OO logic.
- Based on JDBC 'under the hood'.
- No need to deal with the database implementation.
- Entities based on business concepts rather than database structure.
- Transaction management and automatic key generation
- Fast development of application.

ORM Solution

An ORM solution consists of the following entities:

- An API to perform basic CRUD operation on objects of persistent classes.
- A language or API to specify queries that refer to classes and properties of classes.
- A configurable facility for specifying mapping metadata.
- A technique to interact with transactional object to perform dirty checking, lazy association fetching and other optimization function.

Java ORM Frameworks

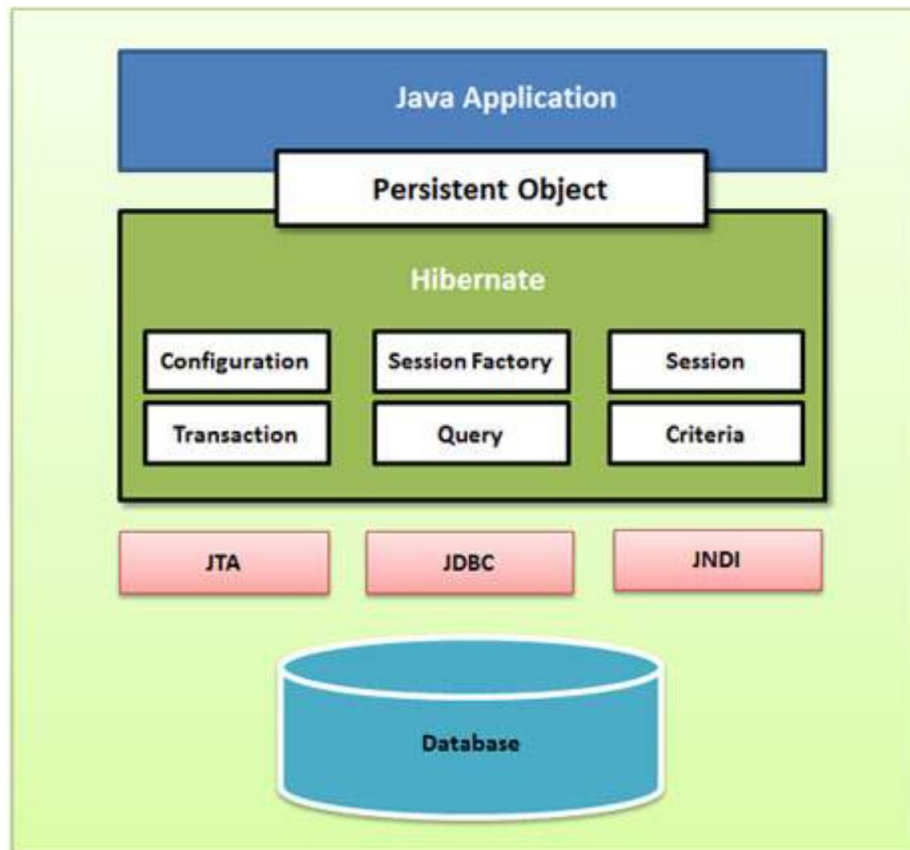
- Enterprise JavaBeans Entity Beans
- Java Data Objects
- Castor
- Toplink
- Spring DAO
- Hibernate etc.

Hibernate

Hibernate is an Object – Relational Mapping (ORM) solution for java. It is an open source persistent framework created by Gavin King in 2001. It is powerful, high performance Object – Relational Persistence and Query service for any data java application.

Hibernate maps java classes to database tables and from java data types to SQL data types and relieves the developer from 95% of common data persistence related programming tasks. Hibernate lies between traditional java objects and database server to handle all the work in persisting those objects based on the appropriate O/R mechanism and patterns.

Hibernate Architecture



Hibernate has a layered architecture which helps the user to operate without having to know the underlying APIs. Hibernate makes use of the database and configuration data to provide persistence services (and persistent objects) to the application.

Hibernate uses various existing java APIs, like JDBC, Java Transaction API (JTA) and Java Naming and Directory Interface (JNDI). JDBC provides a rudimentary level of abstraction of functionality common to relational databases, allowing almost any database with a JDBC driver to be supported by Hibernate. JNDI and JTA allow Hibernate to be integrated with J2EE application servers.

Following section gives brief description of each of the class object involved in Hibernate Application Architecture.

Configuration Object

The configuration object is the first Hibernate object you create in any Hibernate application. It is created only once during application initialization. The configuration object provides two key components

- **Database Connection:** This is handled through one or more configuration files supported by Hibernate. These files are **hibernate.properties** and **hibernate.cfg.xml**.

- **Class Mapping Setup:** This component creates the connection between the java classes and database tables.

SessionFactory Object

SessionFactory Object configures Hibernate for the application using the supplied configuration file and allows for a session object to be instantiated. The sessionFactory is a thread safe object and used by all the threads of an application. The SessionFactory is a heavyweight object; it is usually created during application startup and kept for later use.

Session Object

A session is used to get a physical connection with a database. The session object is lightweight and designed to be instantiated each time an interaction is needed with the database. Persistent objects are saved and retrieved through a Session object. The session objects should not be kept open for a long time because they are not usually thread safe and they should be created and destroyed as needed.

Transaction Object

A transaction represents a unit of work with the database and most of the RDBMS supports transaction functionality. Transactions in Hibernate are handled by an underlying transaction manager and transaction (From JDBC or JTA). This is an optional object and Hibernate applications may choose not to use this interface, instead managing transactions in their own application code.

```
StandardServiceRegistry ssr = new
StandardServiceRegistryBuilder().configure("hibernate.cfg.xml").build();
Metadata meta = new MetadataSources(ssr).getMetadataBuilder().build();

// Create the SessionFactory Instance
SessionFactory factory = meta.getSessionFactoryBuilder().build();

// Create the session
Session session = factory.openSession();

// Create the transaction
Transaction t = session.beginTransaction();
```

Query Object

Query object use SQL or Hibernate Query Language (HQL) string to retrieve data from the database and create objects. A query instance is used to bind query parameters, limit the number of results returned by the query and finally to execute the query.

Criteria Object

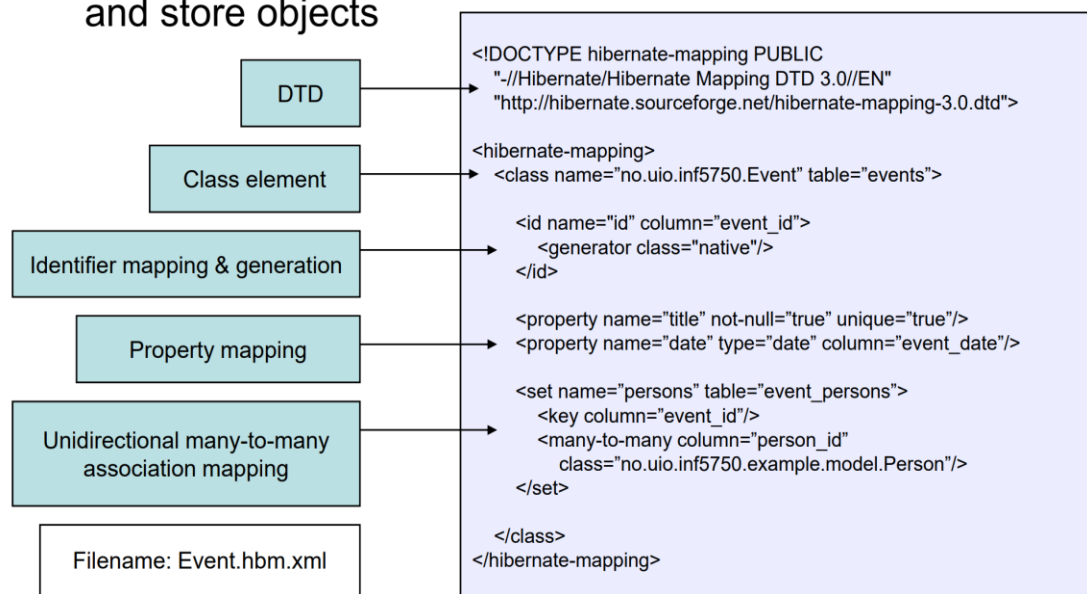
Criteria objects are used to create and execute object oriented criteria queries to retrieve objects.

Advantage of Hibernate

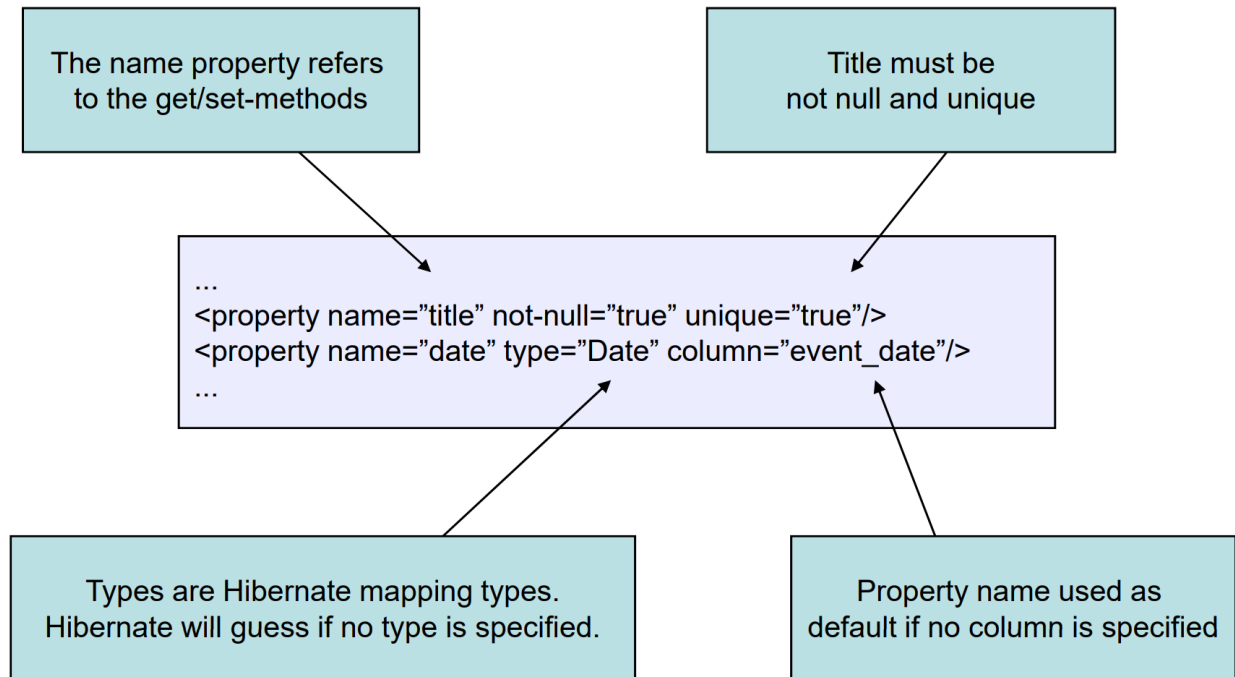
- Hibernate takes care of mapping java classes to database table using XML files and without writing any line of code.
- Provides simple APIs for storing and retrieving java objects directly to and from the database.
- If there is change in the database or in any table, then you need to change the XML file properties only.
- Abstracts away the unfamiliar SQL types and provides a way to work around familiar java objects.
- Hibernate does not require an application server to operate. Manipulates Complex associations of objects of your database.
- Minimizes database access with smart fetching strategies.
- Provides simple querying of data.

Hibernate Mapping files

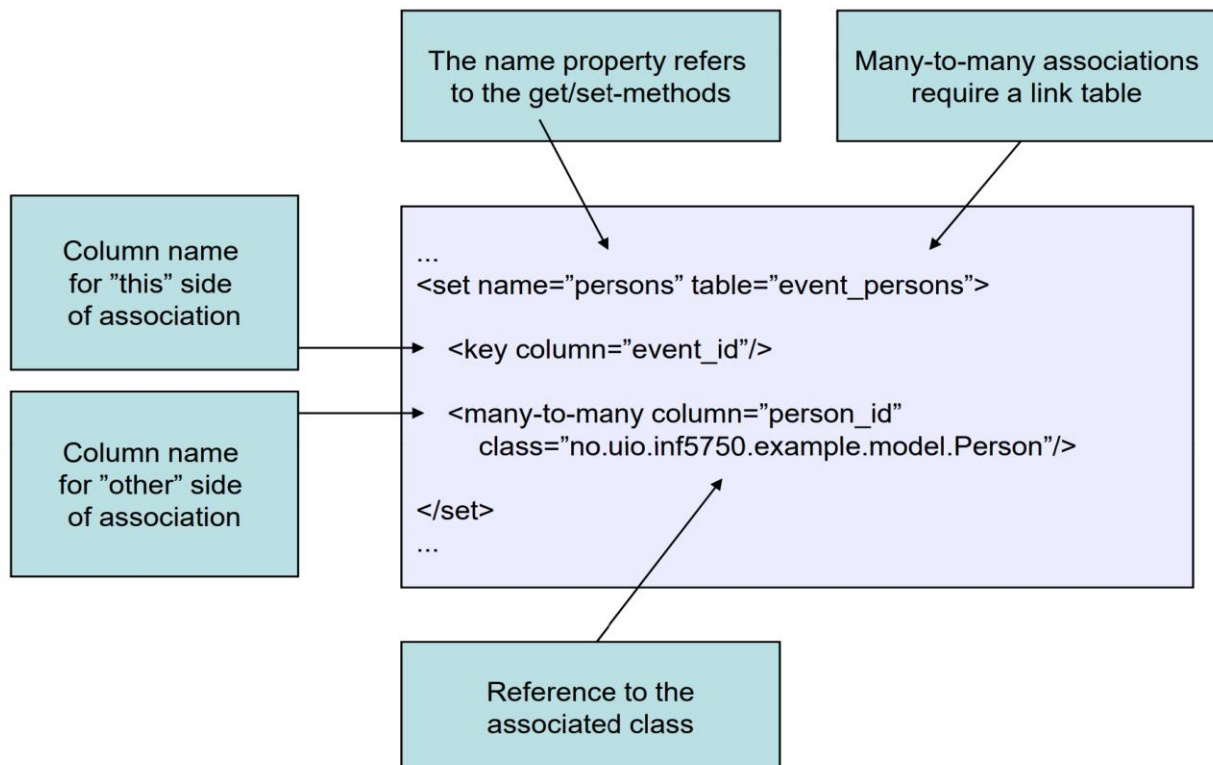
- Tells Hibernate which tables and columns to use to load and store objects



Property Mapping



Association Mapping



Java Web Framework

1. Spring

Spring is a powerful, lightweight, and most popular framework which makes Java quicker, easier, and safer to use. This framework is very popular among developers for its speed, simplicity, and productivity which helps to create enterprise-level web applications with complete ease. Spring MVC and Spring Boot made Java modern, reactive, and cloud-ready to build high-performance complex web applications hence used by many tech giants, including Netflix, Amazon, Google, Microsoft, etc.

- Using Spring's flexible and comprehensive third-party libraries you can build any web application you can imagine.
- Start a new Spring project in seconds, with, fast startup, fast shutdown, and optimized execution by default.
- Spring provides a lightweight container that can be triggered without a web server or application server.
- It provides backward compatibility and easy testability of your project.
- It supports JDBC which improves productivity and reduces error as much as possible
- It supports modularity and both XML and annotation-based configuration
- Spring Boot has a huge ecosystem and community with extensive documentation and multiple Spring tutorials

2. Grails

It is a dynamic **full-stack Java framework** based on the MVC design pattern. which is easy to learn and most suitable for beginners. Grails is an Object Oriented Language that enhances developer productivity. While written in Groovy it can run on the Java platform and is perfectly compatible with Java syntax.

- Easy to Create tags for the View,
- built-in support for RESTFUL APIs.
- you can mix Groovy and Java using Grails,
- best suitable for Rapid Development,
- configuration features are dynamic, no need to restart the server

3. Google Web Toolkit (GWT)

It is a very popular open-source Java framework used by a large number of developers around the world for building and optimizing complex browser-based applications. This framework is used for the productive development of high-performance complex web applications without being an expert in front-end technologies like JavaScript or responsive design. It converts Java code into JavaScript code which is a remarkable feature of GWT. Popular Google applications like AdSense and AdWords are written and using this framework.

- Google APIs are vastly used in GWT applications.
- Open-source and Developer-friendly.
- Easily create beautiful UIs without vast knowledge in front-end scripting languages.
- Create optimized web applications that are easy to debug.
- Compiles the Java source code into JavaScript files that can run on all major browsers.

4. Struts

It is a very useful framework, an open-source MVC framework for creating modern enterprise-level Java web applications that favor convention over configuration and reduce overall development time. It comes with plugins to support REST, AJAX and JSON and can be easily integrated with other Java frameworks like Spring and Hibernate.

- Super Flexible and beginner-friendly,
- Reliable, based on MVC design pattern.
- Integration with REST, JSON, and AJAX.
- Creative themes and templates make development tasks faster.
- Extend capabilities for complex development,
- Reduced development time and Effort, makes dev easier and fun.

5. Java Server Faces (JSF)

It is quite similar to Struts, a free web application developed framework, maintained by Oracle technology which simplifies building user interfaces for Server-side applications by assembling reusable UI components on a page. JSF is a component-based MVC framework that encapsulates various client-side technologies and focuses more on the presentation layer to allow web developers to create UI simply by just drag and dropping.

- Rich libraries and reusable UI components,
- Easy front-end tools to use without too much coding,
- Jsf helps to improve productivity and consistency,
- Enrich the user experience by adding Ajax events for validations and method invocations.
- It provides an API to represent and manage UI components and instead of using Java, JSF uses XML for view handling.

6. Hibernate

A stable, lightweight ORM Java framework that can easily communicate with any database and is more convenient when working with multiple databases. Working with Hibernate is fun using powerful APIs and several useful tools like Mapping Editor, Wizards, and Reverse Engineering. Many big companies including Platform, DAILY HOTEL, IBM, and Dell use Hibernate in their tech stacks,

- Light-weight and easy to scale up, modify, and configure.
- Complex data manipulation with less coding.
- High productivity and portability,
- Used for RDBMS as well as the NoSQL database.
- Awesome Command-line tools and IDE plugins to make your experience pleasant.

Session Object

A session is used to get a physical connection with a database. The session object is lightweight and designed to be instantiated each time an interaction is needed with the database. Persistent objects are saved and retrieved through a Session object. The session objects should not be kept

open for a long time because they are not usually thread safe and they should be created and destroyed them as needed.

Introduction to Spring Boot

Spring is widely used for creating scalable application. For web applications, Spring provides Spring MVC, a commonly used module for building robust web applications. The major drawback of traditional Spring projects is that configuration can be time – consuming and overwhelming for new developers. Making an application production – ready requires significant effort.

The solution to this is “Spring Boot”. Spring Boot is built on the top of the Spring Framework and includes all Spring features while simplifying configuration. It has become a developer favorite due to its rapid development capabilities, allowing developers to focus on business logic instead of struggling with configurations.

Spring Boot is a microservice – based framework that enables quick development of production – ready applications. A prerequisite for learning Spring Boot is a basic understanding of the Spring Framework.

Features of Spring Boot

1. Auto – Configuration

Spring Boot **eliminates the need for heavy XML configuration**, which is common in traditional Spring MVC projects. Instead, everything is auto-configured. Developers only need to add the appropriate configuration to utilize specific functionalities. For example, if we want to use Hibernate (ORM) we can simply add the **@Table** annotation to our model/entity class and the **@Column** annotation to map it to database tables and columns. By 2025, Spring Boot's auto-configuration has become even smarter, utilizing AI-driven optimizations to further reduce manual setup.

2. Easy Maintenance and Creation of REST Endpoints

Creating REST APIs is incredibly easy in Spring Boot. With annotations like **@RestController** and **@RequestMapping**, developers can quickly define endpoints. By 2025, Spring Boot has introduced even more advanced annotations like **@GetMapping**, **@PostMapping**, **@PutMapping**, and **@DeleteMapping**, making REST API development more intuitive and efficient

3. Embedded Tomcat Server

Unlike traditional Spring MVC projects, where you need to manually install and configure a Tomcat server, Spring Boot comes with an Embedded Tomcat Server. This allows applications to be hosted directly without additional setup. By 2025, Spring Boot also supports other

embedded servers like Jetty and Undertow, giving developers more flexibility based on their application requirements.

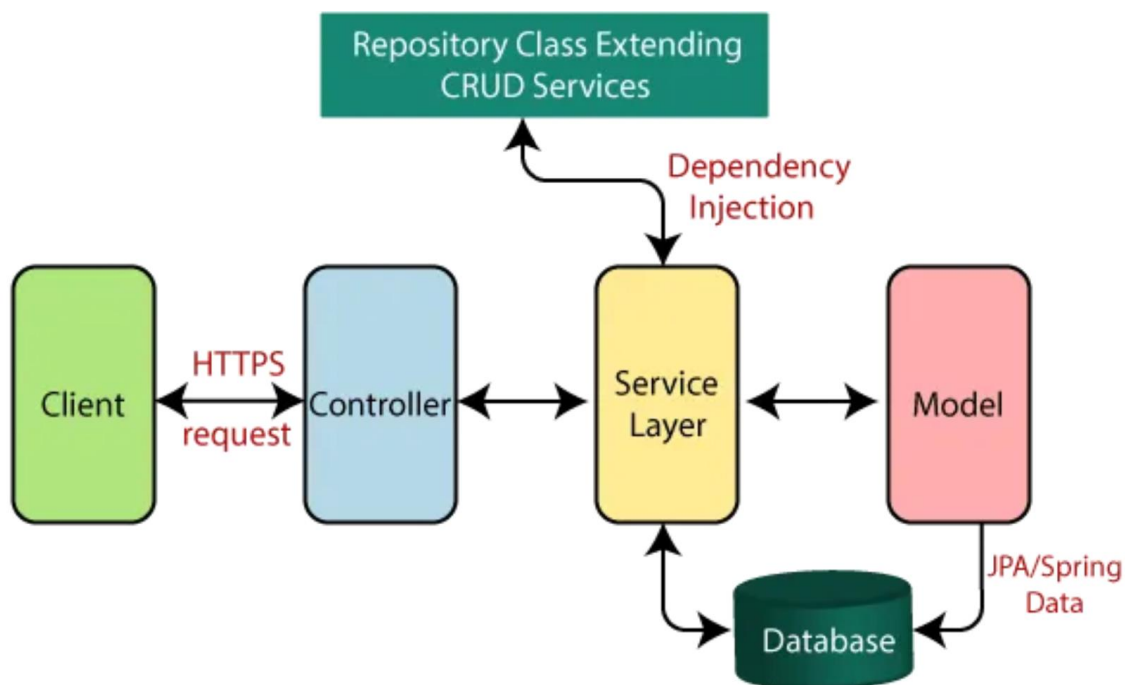
4. Easy Deployment

Spring Boot simplifies deployment by allowing applications to be packaged as JAR or WAR files. These files can be directly deployed to a Tomcat server or cloud environment. By 2025, Spring Boot has enhanced its deployment capabilities with seamless integration into Kubernetes and Docker, making it easier to deploy and scale applications in cloud-native environments.

5. Microservice – Based Architecture

Spring Boot is designed for microservices, which are small, independent modules that focus on a single functionality. For example, in a hospital management system, you might have separate services for patient registration, billing, and database management. By 2025, Spring Boot has further optimized its microservice support with features. In a monolithic system, all features are bundled into a single codebase, making it difficult to maintain and scale. In a microservice-based system, each feature is divided into smaller, independent services. This modular approach makes the system easier to maintain, debug, and deploy. Each service can be built using different technologies suited to its specific requirements.

Spring Boot Architecture



To understand the architecture of Spring Boot, Let's examine its different layers and components.

Layers in Spring Boot

Spring Boot follows a layered architecture with the following key layers.

Client Layer

- This represents the external system or user that interacts with the application by sending HTTPs requests.

Controller Layer (Presentation Layer)

- Handles incoming HTTP requests from the client.
- Processes the request and sends a response.
- Delegates business logic processing to the service layer.

Service Layer (Business Logic Layer)

- Contains business logic and service classes.
- Communicates with the repository layer to fetch or update data.
- Uses Dependency injection to get required repository services.

Repository Layer (Data Access Layer)

- Handles CRUD (create, read, update, delete) operation on the database.
- Extends Spring Data JPA or other persistence mechanisms.

Model Layer (Entity Layer)

- Represents database entities and domain models.
- Maps to tables in the database using JPA/Spring Data

Database Layer

- The actual database that stores application data.
- Spring Boot interacts with it through JPA / Spring Data

Multithreading

A thread is a single sequential flow of control within a program. Threads are also called light weight processes. A program that has many threads running concurrently is called multithreading. Java multithreading program is platform dependent. Thus, a multithreaded application could behave differently on different java implementations.

By definition multitasking is when multiple processes share common processing resources such as a CPU. Multi-threading extends the idea of multitasking into applications where we can subdivide specific operations within a single application into individual threads. Each of the threads can run in parallel. The OS divides processing time not only among different applications, but also among

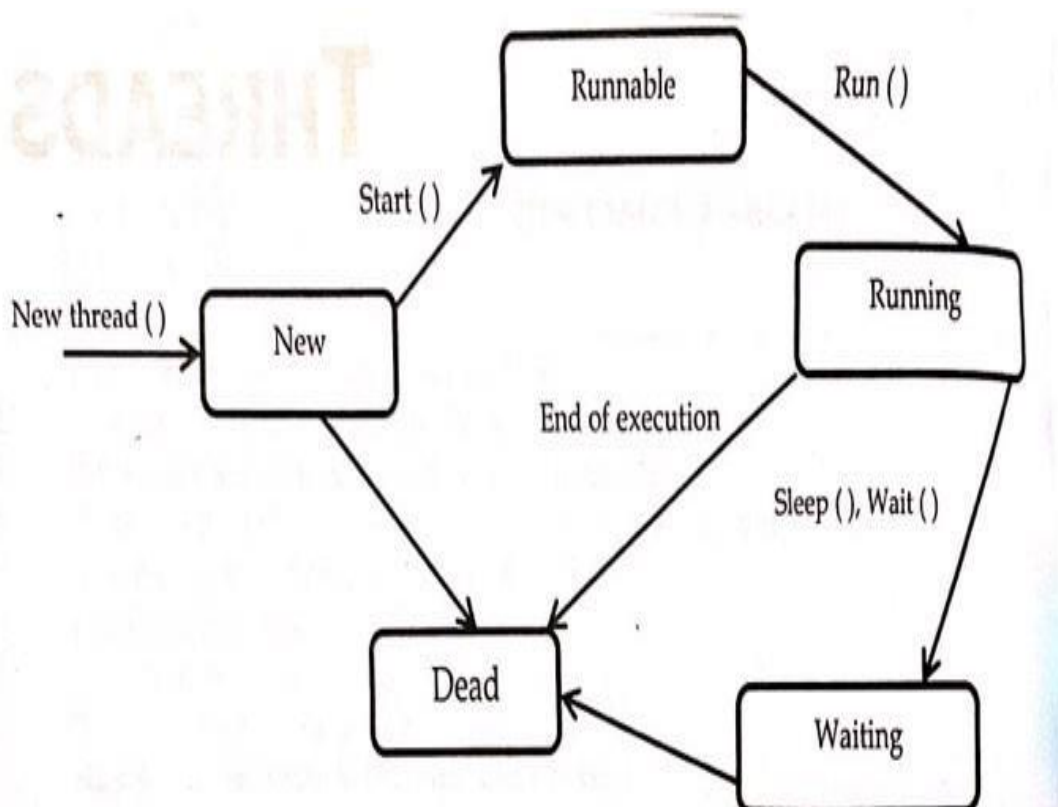
each thread within an application. Multi-threading enable us to write in a way where multiple activities can proceed concurrently in the same program.

Transition Diagram or State Diagram or Life Cycle of a Thread

A thread can be in one of the five states. According to sun, there is only 4 states in thread life cycle in java new, runnable, non-runnable and terminated. There is no running state. But for better understanding the threads, we are explaining it in the 5 states.

The life cycle of the thread in java is controlled by JVM. The java thread states are as follows:

1. Newborn
2. Runnable
3. Running
4. Non-Runnable (Blocked)
5. Terminated or Dead



1. **New**

The thread is in new state if you create an instance of Thread class but before the invocation of start() method. It remains in this state until the program starts the thread. It is also referred to as a born thread.

2. **Runnable**

The thread is in runnable state after invocation of start() method, but the thread scheduler has not selected it to be the running thread. A thread in this state is considered to be executing its task.

3. **Running**

The thread is in running state if the thread scheduler has selected it. Sometimes, a thread transitions to the waiting state while the thread waits for another thread to perform a task. A thread transitions back to the runnable state only when another thread signals the waiting thread to continue executing.

4. **Non – Runnable (Blocked)**

This is the state when the thread is still alive, but is currently not eligible to run. A runnable thread can enter the timed waiting state for a specified interval of time. This happens when the thread performs some kind of input output action or communicating between the Threads. A thread in this state transitions back to the runnable state when that time interval expires or when the event it is waiting for occurs.

5. **Terminated or Dead**

A runnable thread enters the terminated state when it completes its task or otherwise terminates.

How to create a thread?

Threads are implemented in the form of objects that contain the method called run() method. The run() method should be invoked by an object of the concerned thread. This can be achieved by creating the thread and initiating it with the help of another thread method called start() method. The run() method can be written in two ways.

- By extending Thread class
- By implementing Runnable Interface

Thread Creation by extending the Thread class

We create a class that extends the **java.lang.Thread** class. This class overrides the **run()** method available in the Thread class. A thread begins its life inside **run()** method. We create an object of our new class and call **start()** method to start the execution of a thread. **Start()** invokes the **run()** method on the **Thread** object.

```
class A extends Thread
{
    public void run()
    {
        for(int i=1; i<=50; i++)
        {
            System.out.println(x:"Pokhara");
        }
    }
}

class B extends Thread
{
    public void run()
    {
        for(int i=1; i<=50; i++)
        {
            System.out.println(x:"University");
        }
    }
}

public class threads
{
    Run | Debug
    public static void main(String args[])
    {
        A ob1 = new A();
        B ob2 = new B();

        ob1.start();
        ob2.start();
    }
}
```

Thread Creation by implementing the Runnable Interface

```
class A implements Runnable
{
    public void run()
    {
        for(int i=1; i<=50; i++)
        {
            System.out.println(x:"Pokhara");
        }
    }
}

class B implements Runnable
{
    public void run()
    {
        for(int i=1; i<=100; i++)
        {
            System.out.println(x:"University");
        }
    }
}

public class threads1
{
    Run | Debug
    public static void main(String args[])
    {
        Runnable ob1 = new A();
        Runnable ob2 = new B();

        Thread t1 = new Thread(ob1);
        Thread t2 = new Thread(ob2);

        t1.start();
        t2.start();
    }
}
```

Synchronization

At times when more than one thread try to access a shared resource, we need to ensure that resource will be used by only one thread at a time. The process by which this is achieved is called synchronization. The synchronization keyword in java creates a block of code referred to as critical section.

```
Synchronized(object)
{
    // statement to be synchronized
}
```

Every java object with a critical section of code gets a lock associated with the object. To enter critical section a thread need to obtain the corresponding object's lock. If we do not use synchronization, and let two or more threads access a shared resource at the same time, it will lead to distorted results.

Synchronization is broadly classified into two categories:

1. Method level synchronization
2. Block level synchronization

1. Method level synchronization

Using synchronized methods is a way to accomplish synchronization.

Example

```
public synchronized void run()
{
    // same resource
}
t1.start();
t2.start();
t3.start();
```

2. Block level synchronization

If we have to synchronize access to an object of a class or we only want a part of a method to be synchronized to an object then we can use synchronized block for it.

Example:

```
public void run()
{
    synchronized(this)
    {
        // same resource
    }
}
t1.start();
t2.start();
t3.start();
```

When we use synchronized keyword with a method, it acquires a lock in the object for the whole method. It means that no other thread can use any synchronized method until the current thread, which has invoked its synchronized method has finished its execution.

Synchronization block acquires a lock in the object only between parentheses after the synchronized keyword. This means that no other thread can acquire a lock on the locked object until the synchronized block exist. But other threads can access the rest of the code of the method.

Example

```
class Shared
{
    // Synchronized method (method-level synchronization)
    synchronized void print(String msg)
    {
        for (int i = 1; i <= 2000; i++)
        {
            System.out.println(msg);
        }
    }
}

class A extends Thread
{
    Shared s;
    A(Shared s)
    {
        this.s = s;
    }

    public void run()
    {
        s.print(msg: "Pokhara");
    }
}

class B extends Thread
{
    Shared s;
    B(Shared s)
    {
        this.s = s;
    }

    public void run()
    {
        s.print(msg: "University");
    }
}

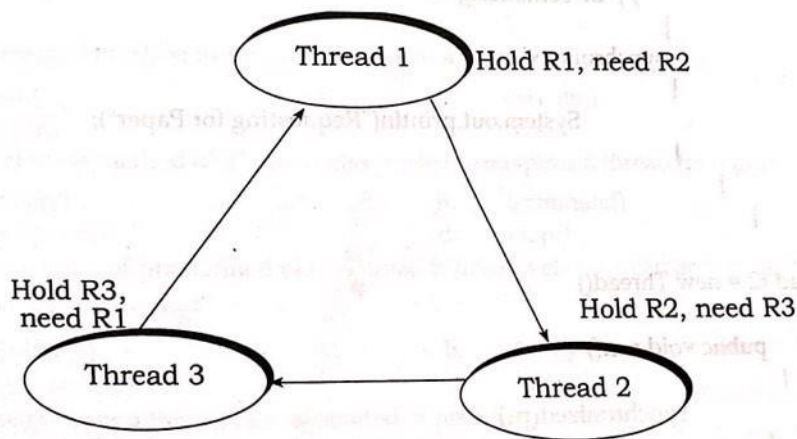
public class methodsyn
{
    Run | Debug
    public static void main(String args[])
    {
        Shared sharedObj = new Shared(); // common shared object

        A ob1 = new A(sharedObj);
        B ob2 = new B(sharedObj);

        ob1.start();
        ob2.start();
    }
}
```

Thread Deadlock

Deadlock is a situation of complete lock, when no thread can complete its execution because lack of resources. In the below picture, thread 1 is holding a resource R1, and need another resource R2 to finish execution, but R2 is locked by thread 2, which needs R3, which in turn is locked by thread 3. hence none of them can finish and are stuck in a deadlock.



Thread Pooling

A thread pool is a collection of pre – initialized threads. The general plan behind a thread pool is to form variety of threads at method startup and place them into a pool, wherever they sit and expect work. Once a server receives a call for participation, it awakens a thread from this pool, if one is available and passes it the request for service. Once the thread completes its service, it returns to the pool and awaits a lot off work. If the pool contains no accessible thread, the server waits till one become free.

Why use thread pools in java

- It saves time as a result of there is no need to produce new thread.
- It utilized in servlet and JSP where ever instrumentality creates a thread pool to method the request.

Creating thread pools in java

Java provides a **java.util.concurrent.Executors** class provides couple of methods to create a thread pools.

Following are methods to create a thread pools

- public static ExecutorService newCachedThreadPool()**
- public static ExecutorService newFixedThreadPool(int n Threads)**
- public static ScheduledExecutorService newScheduleThreadPool(int corePoolSize)**
- public static ExecutorService newWorkStealingPool()**

Chapter 1

Introduction to Design Patterns

1.1 Introduction

In the late 70's, an architect named Christopher Alexander started the concept of patterns. Alexander's work focused on finding patterns of solutions to particular sets of forces within particular contexts.

Christopher Alexander was a civil engineer and an architect, his patterns were related to architects of buildings, but the work done by him inspired an interest in the object-oriented (OO) community, and a number of innovators started developing patterns for software design. Kent Beck and Ward Cunningham were among the few who presented the set of design patterns for Smalltalk in an OOPSLA conference. James Coplien was another who actively promoted the tenets of patterns.

Soon, the patterns community started growing at OOPSLA, as it placed an environment for the members to share their innovations and ideas about the patterns. Another important forum for the evolution of the patterns movement was the Hillside Group, established by Kent Beck and Grady Booch.

This is what design patterns are - the distillation of expertise by an exuberant and robust community. This is crowd sourcing at its best. The patterns community that has grown over the decade-plus since the original GoF work is large and energetic. Grady Booch and Celso Gonzalez have been collecting every pattern they can find in the industry. So far, they have over 2,000 of them.

This course is all about Design Patterns. In this course, we will present to you, the most useful and famous design patterns. In this lesson, first we will see what really are the Design Patterns. What is their use? Why one should really use them, and how to use them?

Later, we will also see how patterns are organized, and categorized into different groups according to their behavior and structure.

In the next several lessons, we will discuss about the different design patterns one by one. We will go into depth and analyze each and every design pattern, and will also see how to implement them in Java.

1.2 What are Design Patterns

As an Object Oriented developer, we may think that our code contains all the benefits provided by the Object Oriented language. The code we have written is flexible enough that we can make any changes to it with less or any pain. Our code is re-usable so that we can re-use it anywhere without any trouble. We can maintain our code easily and any changes to a part of the code will not affect any other part of the code.

Unfortunately, these advantages do not come by its own. As a developer, it is our responsibility to design the code in such a way which allow our code to be flexible, maintainable, and re-usable.

Designing is an art and it comes with the experience. But there are some set of solutions already written by some of the advanced and experienced developers while facing and solving similar designing problems. These solutions are known as Design Patterns.

The Design Patterns is the experience in designing the object oriented code.

Design Patterns are general reusable solution to commonly occurring problems. These are the best practices, used by the experienced developers. Patterns are not complete code, but it can use as a template which can be applied to a problem. Patterns

are re-usable; they can be applied to similar kind of design problem regardless to any domain. In other words, we can think of patterns as a formal document which contains recurring design problems and its solutions. A pattern used in one practical context can be re-usable in other contexts also.

Christopher had said that “Each pattern describes a problem that occurs over and over again in our environment, and then describes the core of the solution to that problem, in such a way that you can use this solution a million times over, without ever doing it the same way twice”.

In general, a pattern has four essential elements:

- **Pattern name**, is used to provide a single and meaningful name to the pattern which defines a design problem and a solution for it. Naming a design pattern helps itself to be referred to others easily. It also becomes easy to provide documentation for and the right vocabulary word makes it easier to think about the design.
- **The problem describes when to apply the pattern**. It explains the problem and its context. It might describe specific design problems such as how to represent algorithms as objects. It might describe a class or object structures that are symptomatic of an inflexible design. Sometimes the problem will include a list of conditions that must be met before it makes sense to apply the pattern.
- The **solution** describes the elements that make up the design, their relationships, responsibilities, and collaborations. The solution is not the complete code, but it works as a template which can be fulfilled with code. Instead, the pattern provides an abstract description of a design problem and how a general arrangement of elements (classes and objects in our case) solves it.
- The **results and consequences** of applying the pattern. The consequences for software often concern space and time trade-offs. They may address language and implementation issues as well. Since reuse is often a factor in object-oriented design, the consequences of a pattern include its impact on a system’s flexibility, extensibility, or portability. Listing these consequences explicitly helps you understand and evaluate them.

1.3 Why use them

Flexibility: Using design patterns your code becomes flexible. It helps to provide the correct level of abstraction due to which objects become loosely coupled to each other which makes your code easy to change.

Reusability: Loosely coupled and cohesive objects and classes can make your code more reusable. This kind of code becomes easy to be tested as compared to the highly coupled code.

Shared Vocabulary: Shared vocabulary makes it easy to share your code and thought with other team members. It creates more understanding between the team members related to the code.

Capture best practices: Design patterns capture solutions which have been successfully applied to problems. By learning these patterns and the related problem, an inexperienced developer learns a lot about software design.

Design patterns make it easier to reuse successful designs and architectures.

Expressing proven techniques as design patterns makes them more accessible to developers of new systems. Design patterns help you choose design alternatives that make a system reusable and avoid alternatives that compromise reusability. Design patterns can even improve the documentation and maintenance of existing systems by furnishing an explicit specification of class and object interactions and their underlying intent. Put simply, design patterns help a designer get a design "right" faster.

1.4 How to select and use one

There are a number of design patterns to choose from; to choose one, you must have good knowledge of each one of them. There are many design patterns which look very similar to one another. They solve almost a similar type of design problem and also have similar implementation. One must have a very deep understanding of them in order to implement the correct design pattern for the specific design problem.

First, you need to identify the kind of design problem you are facing. A design problem can be categorized into creational, structural, or behavioral. Based to this category you can filter the patterns and selects the appropriate one. For example:

- **There are too many instances of a class which represent only a single thing, the value in the properties of the objects are same, and they are only used as read-only:** you can select the Singleton pattern for this design problem which ensures only a single instance for the entire application. It also helps to decrease the memory size.
- **Classes are too much dependent on each other. A Change in one class affects all other dependent classes:** you can use Bridge, Mediator, or Command to solve this design problem.
- **There are two different incompatible interfaces in two different parts of the code, and your need is to convert one interface into another which is used by the client code to make the entire code work:** the Adapter pattern fits into this problem.

A design pattern can be used to solve more than one design problem, and one design problem can be solved by more than one design patterns. There could be plenty of design problems and solutions for them, but, to choose the pattern which fits exactly is depends on your knowledge and understanding about the design patterns. It also depends on the code you already have in place.

1.5 Categorization of patterns

Design patterns can be categorized in the following categories:

- Creational patterns
- Structural patterns
- Behavior patterns

1.5.1 Creational patterns

Creational design patterns are used to design the instantiation process of objects. The creational pattern uses the inheritance to vary the object creation.

There are two recurring themes in these patterns. First, they all encapsulate knowledge about which concrete classes the system uses. Second, they hide how instances of these classes are created and put together. All the system at large knows about the objects is their interfaces as defined by abstract classes. Consequently, the creational patterns give you a lot of flexibility in what gets created, who creates it, how it gets created, and when.

There can be some cases when two or more patterns looks fit as a solution to a problem. At other times, the two patterns complement each other for example; Builder can be used with other pattern to implements which components to get built.

1.5.2 Structural patterns

Structural patterns are concerned with how classes and objects are composed to form larger structures. Structural class patterns use inheritance to compose interfaces or implementations. As a simple example, consider how multiple inheritance mixes two or more classes into one. The result is a class that combines the properties of its parent classes. This pattern is particularly useful for making independently developed class libraries work together.

Rather than composing interfaces or implementations, structural object patterns describe ways to compose objects to realize new functionality. The added flexibility of object composition comes from the ability to change the composition at run-time, which is impossible with static class composition.

1.5.3 Behavior patterns

Behavioral patterns are concerned with algorithms and the assignment of responsibilities between objects. Behavioral patterns describe not just patterns of objects or classes but also the patterns of communication between them. These patterns characterize complex control flow that's difficult to follow at run-time. They shift your focus away from flow of control to let you concentrate just on the way objects are interconnected.

Behavioral object patterns use object composition rather than inheritance. Some describe how a group of peer objects cooperate to perform a task that no single object can carry out by itself. An important issue here is how peer objects know about each other. Peers could maintain explicit references to each other, but that would increase their coupling. In the extreme, every object would know about every other. The Mediator pattern avoids this by introducing a mediator object between peers. The mediator provides the indirection needed for loose coupling.

The below tables shows the list of patterns under their respective categories:

Table 1.1: List of patterns

Creational Patterns	Structural Patterns	Behavioral Patterns
Abstract Factory	Adapter	Chain of Responsibility
Builder	Bridge	Command
Factory Method	Composite	Interpreter
Prototype	Decorator	Iterator
Singleton	FaA§ade	Mediator
	Flyweight	Memento
	Proxy	Observer
		State
		Strategy
		Template Method
		Visitor

Chapter 6

Singleton Design Pattern

6.1 Singleton Pattern

Sometimes it's important for some classes to have exactly one instance. There are many objects we only need one instance of them and if we, instantiate more than one, we'll run into all sorts of problems like incorrect program behavior, overuse of resources, or inconsistent results.

You may require only one object of a class, for example, when you are creating the context of an application, or a thread manageable pool, registry settings, a driver to connect to the input or output console etc. More than one object of that type clearly will cause inconsistency to your program.

The Singleton Pattern ensures that a class has only one instance, and provides a global point of access to it. However, although the Singleton is the simplest in terms of its class diagram because there is only one single class, its implementation is a bit trickier.



Figure 6.1: screenshot

In this lesson, we will try different ways to create only a single object of the class and will also see how one way is better than the other.

6.2 How to create a class using the Singleton Pattern

There could be many ways to create such type of class, but still, we will see how one way is better than the other.

Let's start with a simple way.

What if, we provide a global variable that makes an object accessible? For example:

```
package com.javacodegeeks.patterns.singletonpattern;

public class SingletonEager {
    public static SingletonEager sc = new SingletonEager();
}
```

As we know, there is only one copy of the static variables of a class, we can apply this. As far as, the client code is using this `sc` static variable its fine. But, if the client uses a new operator there would be a new instance of this class.

To stop the class to get instantiated outside the class, let's make the constructor of the class as private.

```
package com.javacodegeeks.patterns.singletonpattern;

public class SingletonEager {
    public static SingletonEager sc = new SingletonEager();
    private SingletonEager() {}
}
```

Is this going to work? I think yes. By keeping the constructor private, no other class can instantiate this class. The only way to get the object of this class is using the `sc` static variable which ensures only one object is there.

But as we know, providing a direct access to a class member is not a good idea. We will provide a method through which the `sc` variable will get access, not directly.

```
package com.javacodegeeks.patterns.singletonpattern;

public class SingletonEager {
    private static SingletonEager sc = new SingletonEager();
    private SingletonEager() {}
    public static SingletonEager getInstance() {
        return sc;
    }
}
```

So, this is our singleton class which makes sure that only one object of the class gets created and even if there are several requests, only the same instantiated object will be returned.

The one problem with this approach is that, the object would get created as soon as the class gets loaded into the JVM. If the object is never requested, there would be an object useless inside the memory.

It's always a good approach that an object should get created when it is required. So, we will create an object on the first call and then will return the same object on other successive calls.

```
package com.javacodegeeks.patterns.singletonpattern;

public class SingletonLazy {

    private static SingletonLazy sc = null;
    private SingletonLazy() {}
    public static SingletonLazy getInstance() {
        if(sc==null){
            sc = new SingletonLazy();
        }
        return sc;
    }
}
```

In the `getInstance()` method, we check if the static variable `sc` is null, then instantiate the object and return it. So, on the first call when `sc` would be null the object gets created and on the next successive calls it will return the same object.

This surely looks good, doesn't it? But this code will fail in a **multi-threaded environment**. Imagine two threads concurrently accessing the class, thread `t1` gives the first call to the `getInstance()` method, it checks if the static variable `sc` is null and then gets interrupted due to some reason. Another thread `t2` calls the `getInstance()` method successfully passes the if check and instantiates the object. Then, thread `t1` gets awake and it also creates the object. At this time, there would be two objects of this class.

To avoid this, we will use the `synchronized` keyword to the `getInstance()` method. With this way, we force every thread to wait its turn before it can enter the method. So, no two threads will enter the method at the same time. The `synchronized`

comes with a price, it will decrease the performance, but if the call to the `getInstance()` method isn't causing a substantial overhead for your application, forget about it. The other workaround is to move to eager instantiation approach as shown in the previous example.

```
package com.javacodegeeks.patterns.singletonpattern;

public class SingletonLazyMultithreaded {

    private static SingletonLazyMultithreaded sc = null;
    private SingletonLazyMultithreaded() {}
    public static synchronized SingletonLazyMultithreaded getInstance() {
        if(sc==null) {
            sc = new SingletonLazyMultithreaded();
        }
        return sc;
    }
}
```

But if you want to use synchronization, there is another technique known as “double-checked locking” to reduce the use of synchronization. With the double-checked locking, we first check to see if an instance is created, and if not, then we synchronize. This way, we only synchronize the first time.

```
package com.javacodegeeks.patterns.singletonpattern;

public class SingletonLazyDoubleCheck {

    private volatile static SingletonLazyDoubleCheck sc = null;
    private SingletonLazyDoubleCheck() {}
    public static SingletonLazyDoubleCheck getInstance() {
        if(sc==null) {
            synchronized(SingletonLazyDoubleCheck.class) {
                if(sc==null) {
                    sc = new SingletonLazyDoubleCheck();
                }
            }
        }
        return sc;
    }
}
```

Apart from this, there are some other ways to break the singleton pattern.

- If the class is `Serializable`.
- If it's `Clonable`.
- It can be break by Reflection.
- And also if, the class is loaded by multiple class loaders.

The following example shows how you can protect your class from getting instantiated more than once.

```
package com.javacodegeeks.patterns.singletonpattern;

import java.io.ObjectStreamException;
import java.io.Serializable;

public class Singleton implements Serializable{

    private static final long serialVersionUID = -1093810940935189395L;
    private static Singleton sc = new Singleton();
    private Singleton() {
```

```

        if(sc!=null){
            throw new IllegalStateException("Already created.");
        }
    }
    public static Singleton getInstance(){
        return sc;
    }

    private Object readResolve() throws ObjectStreamException{
        return sc;
    }

    private Object writeReplace() throws ObjectStreamException{
        return sc;
    }

    public Object clone() throws CloneNotSupportedException{
        throw new CloneNotSupportedException("Singleton, cannot be cloned");
    }

    private static Class getClass(String classname) throws ClassNotFoundException {
        ClassLoader classLoader = Thread.currentThread().getContextClassLoader();
        if(classLoader == null)
            classLoader = Singleton.class.getClassLoader();
        return (classLoader.loadClass(classname));
    }
}

```

- Implement the `readResolve()` and `writeReplace()` methods in your singleton class and return the same object through them.
- You should also implement the `clone()` method and throw an exception so that the singleton cannot be cloned.
- An "if condition" inside the constructor can prevent the singleton from getting instantiated more than once using reflection.
- To prevent the singleton getting instantiated from different class loaders, you can implement the `getClass()` method. The above `getClass()` method associates the classloader with the current thread; if that classloader is null, the method uses the same classloader that loaded the singleton class.

Although we can use all these techniques, there is one simple and easier way of creating a singleton class. As of JDK 1.5, you can create a singleton class using enums. The Enum constants are implicitly static and final and you cannot change their values once created.

```

package com.javacodegeeks.patterns.singletonpattern;

public class SingletonEnum {

    public enum SingleEnum{
        SINGLETON_ENUM;
    }
}

```

You will get a compile time error when you attempt to explicitly instantiate an Enum object. As Enum gets loaded statically, it is thread-safe. The clone method in Enum is final which ensures that enum constants never get cloned. Enum is inherently serializable, the serialization mechanism ensures that duplicate instances are never created as a result of deserialization. Instantiation using reflection is also prohibited. These things ensure that no instance of an enum exists beyond the one defined by the enum constants.

6.3 When to use Singleton

- There must be exactly one instance of a class, and it must be accessible to clients from a well-known access point.
- When the sole instance should be extensible by sub-classing, and clients should be able to use an extended instance without modifying their code.

6.4 Download the Source Code

This was a lesson on Singleton Pattern. You may download the source code here: [SingletonPattern-Project](#)

Chapter 13

Factory Method Design Pattern

13.1 Introduction

In today's modern world, everyone is using software to facilitate their jobs. Recently, a product company has shifted the way they used to take orders from their clients. The company is now looking to use an application to take orders from them. They receive orders, errors in orders, feedback for the previous order, and responses to the order in an XML format. The company has asked you to develop an application to parse the XML and display the result to them.

The main challenge for you is to parse an XML and display its content to the user. There are different XML formats depending on the different types of messages the company receives from its clients. Like, for example, an order type XML has different sets of xml tags as compared to the response or error XML. But the core job is the same; that is, to display to the user the message being carried in these XMLs.

Although the core job is the same, the object that would be used varies according to the kind of XML the application gets from the user. So, an application object may only know that it needs to access a class from within the class hierarchy (hierarchy of different parsers), but does not know exactly which class from among the set of subclasses of the parent class is to be selected.

In this case, it is better to provide a factory, i.e. a factory to create parsers, and at runtime a parser gets instantiated to do the job, according to the kind of XML the application receives from the user.

The Factory Method Pattern, suited for this situation, defines an interface for creating an object, but let subclasses decide which class to instantiate. Factory Method lets a class defer instantiation to subclasses.

Let us see some more details about the Factory Method Pattern and then we will use it to implement the XML parser for the application.

13.2 What is the Factory Method Pattern

The Factory Method Pattern gives us a way to encapsulate the instantiations of concrete types. The Factory Method pattern encapsulates the functionality required to select and instantiate an appropriate class, inside a designated method referred to as a factory method. The Factory Method selects an appropriate class from a class hierarchy based on the application context and other influencing factors. It then instantiates the selected class and returns it as an instance of the parent class type.

The advantage of this approach is that the application objects can make use of the factory method to get access to the appropriate class instance. This eliminates the need for an application object to deal with the varying class selection criteria.

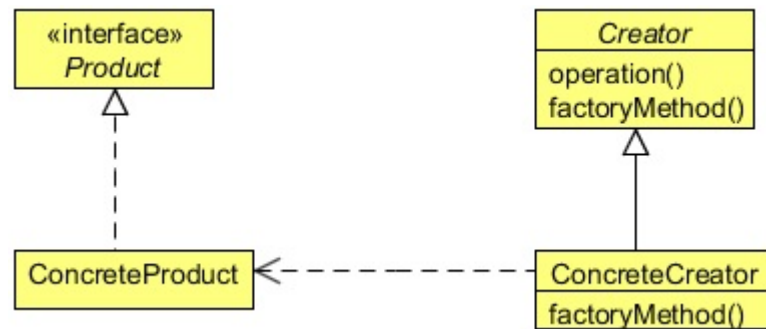


Figure 13.1: screenshot

Product

- Defines the interface of objects the factory method creates.

ConcreteProduct

- Implements the Product interface.

Creator

- Declares the factory method, which returns an object of type Product. Creator may also define a default implementation of the factory method that returns a default ConcreteProduct object.
- May call the factory method to create a Product object.

ConcreteCreator

- Overrides the factory method to return an instance of a ConcreteProduct.

Factory methods eliminate the need to bind application-specific classes into your code. The code only deals with the Product interface; therefore it can work with any user-defined ConcreteProduct classes.

13.3 Implementing Factory Method Pattern

To implement the solution for the application as discussed above, let's first check the product we have.

```
package com.javacodegeeks.patterns.factorymethodpattern;

public interface XMLParser {

    public String parse();

}
```

The above interface will be used by the different XML parsers.

```
package com.javacodegeeks.patterns.factorymethodpattern;

public class ErrorXMLParser implements XMLParser{

    @Override
    public String parse() {
```

```
        System.out.println("Parsing error XML...");
        return "Error XML Message";
    }
}
```

The `ErrorXMLParser` implements the `XMLParser` and is used to parse the error message XMLs.

```
package com.javacodegeeks.patterns.factorymethodpattern;

public class FeedbackXML implements XMLParser{

    @Override
    public String parse() {
        System.out.println("Parsing feedback XML...");
        return "Feedback XML Message";
    }
}
```

The above class is used to parse the feedback message XMLs.

The other XML parsers are:

```
package com.javacodegeeks.patterns.factorymethodpattern;

public class OrderXMLParser implements XMLParser{

    @Override
    public String parse() {
        System.out.println("Parsing order XML...");
        return "Order XML Message";
    }
}

package com.javacodegeeks.patterns.factorymethodpattern;

public class ResponseXMLParser implements XMLParser{

    @Override
    public String parse() {
        System.out.println("Parsing response XML...");
        return "Response XML Message";
    }
}
```

To display the parsed messages from the parsers, an abstract service class is created which will be extended by service specific, i.e. parser specific, display classes.

```
package com.javacodegeeks.patterns.factorymethodpattern;

public abstract class DisplayService {

    public void display(){
        XMLParser parser = getParser();
        String msg = parser.parse();
        System.out.println(msg);
    }

    protected abstract XMLParser getParser();
}
```

```
}
```

The above class is used to display the message fetched by the XML parser to the user. The above class is an abstract class that contains two important methods. The `display` method is used to display the message to the user. The `getParser` method is the factory method which is implemented by the subclasses to instantiate the parser object and the method is used by the `display` method in order to parse the XML and gets the message to display.

Below are the subclasses of the `DisplayService` which implement the `getParser` method.

```
package com.javacodegeeks.patterns.factorymethodpattern;

public class ErrorXMLDisplayService extends DisplayService{

    @Override
    public XMLParser getParser() {
        return new ErrorXMLParser();
    }

}
```

```
package com.javacodegeeks.patterns.factorymethodpattern;

public class FeedbackXMLDisplayService extends DisplayService{

    @Override
    public XMLParser getParser() {
        return new FeedbackXML();
    }

}
```

```
package com.javacodegeeks.patterns.factorymethodpattern;

public class OrderXMLDisplayService extends DisplayService{

    @Override
    public XMLParser getParser() {
        return new OrderXMLParser();
    }

}
```

```
package com.javacodegeeks.patterns.factorymethodpattern;

public class ResponseXMLDisplayService extends DisplayService{

    @Override
    public XMLParser getParser() {
        return new ResponseXMLParser();
    }

}
```

Now, let's test the factory method.

```
package com.javacodegeeks.patterns.factorymethodpattern;

public class TestFactoryMethodPattern {

    public static void main(String[] args) {
```

```
        DisplayService service = new FeedbackXMLDisplayService();
        service.display();

        service = new ErrorXMLDisplayService();
        service.display();

        service = new OrderXMLDisplayService();
        service.display();

        service = new ResponseXMLDisplayService();
        service.display();

    }
}
```

The above class results to the following output:

```
Parsing feedback XML...
Feedback XML Message
Parsing error XML...
Error XML Message
Parsing order XML...
Order XML Message
Parsing response XML...
Response XML Message
```

In the above class, you can clearly see that the by letting the subclasses to implement the factory method creates the different instances of parsers which can be used at runtime according to the need.

13.4 When to use the Factory Method Pattern

Use the Factory Method pattern when

- A class can't anticipate the class of objects it must create.
- A class wants its subclasses to specify the objects it creates.
- Classes delegate responsibility to one of several helper subclasses, and you want to localize the knowledge of which helper subclass is the delegate.

13.5 Factory Method Pattern in JDK

The following are the usage(s) of the Factory Method Pattern in JDK.

- `java.util.Calendar#getInstance()`
- `java.util.ResourceBundle#getBundle()`
- `java.text.NumberFormat#getInstance()`
- `java.nio.charset.Charset#forName()`
- `java.net.URLStreamHandlerFactory#createURLStreamHandler(String)` (Returns singleton object per protocol)

13.6 Download the Source Code

This was a lesson on the Factory Method Pattern. You may download the source code here: [FactoryMethodPattern-Project](#)

Chapter 14

Abstract Factory Method Design Pattern

14.1 Introduction

In the [previous lesson](#), we had developed an application for a product company to parse XMLs and display results to them. We did this by creating different parsers for the different types of communication between the company and its clients. We used the Factory Method Design Pattern to solve their problem.

The application is working fine for them. But now the clients don't want to follow the company's specific XML rules. The clients want to use their own XML rules to communicate with the product company. This means that for every client, the company should have client specific XML parsers. For example, for the NY client there should be four specific types of XML parsers, i.e. NYErrorXMLParser, NYFeedbackXML, NYOrderXMLParser, NYResponseXMLParser, and four different parsers for the TW client.

The company has asked you to change the application according to the new requirement. To develop the parser application we have used the Factory Method Design Pattern in which the exact object to use is decided by the subclasses according to the type of parser. Now, to implement this new requirement, we will use a factory of factories i.e. an Abstract Factory.

This time we need parsers according to client specific XMLs, so we will create different factories for different clients which will provide us the client specific XML to parse. We will do this by creating an Abstract Factory and then implement the factory to provide client specific XML factory. Then we will use that factory to get the desired client specific XML parser object.

Abstract Factory is the design pattern of our choice and before implementing it to solve our problem, let's know more about it.

14.2 What is the Abstract Factory Design Pattern

The Abstract Factory (A.K.A. Kit) is a design pattern which provides an interface for creating families of related or dependent objects without specifying their concrete classes. The Abstract Factory pattern takes the concept of the Factory Method Pattern to the next level. An abstract factory is a class that provides an interface to produce a family of objects. In Java, it can be implemented using an interface or an abstract class.

The Abstract Factory pattern is useful when a client object wants to create an instance of one of a suite of related, dependent classes without having to know which specific concrete class is to be instantiated. Different concrete factories implement the abstract factory interface. Client objects make use of these concrete factories to create objects and, therefore, do not need to know which concrete class is actually instantiated.

The abstract factory is useful for plugging in a different group of objects to alter the behavior of the system. For each group or family, a concrete factory is implemented that manages the creation of the objects and the inter-dependencies and consistency requirements between them. Each concrete factory implements the interface of the abstract factory

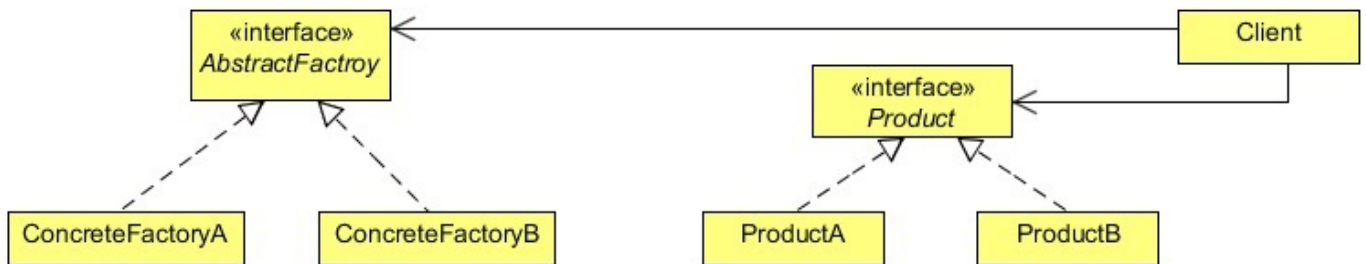


Figure 14.1: screenshot

AbstractFactory

- Declares an interface for operations that create abstract product objects.

ConcreteFactory

- Implements the operations to create concrete product objects.

AbstractProduct

- Declares an interface for a type of product object.

ConcreteProduct

- Defines a product object to be created by the corresponding concrete factory.
- Implements the AbstractProduct interface.

Client

- Uses only interfaces declared by AbstractFactory and AbstractProduct classes.

14.3 Implementing the Abstract Factory Design Pattern

To implement the Abstract Factory Design Pattern will we first create an interface that will be implemented by all the concrete factories.

```

package com.javacodegeeks.patterns.abstractfactorypattern;

public interface AbstractParserFactory {

    public XMLParser getParserInstance(String parserType);

}
  
```

The above interface is implemented by the client specific concrete factories which will provide the XML parser object to the client object. The `getParserInstance` method takes the `parserType` as an argument which is used to get the message specific (error parser, order parser etc) parser object.

The two client specific concrete parser factories are:

```
package com.javacodegeeks.patterns.abstractfactorypattern;

public class NYParserFactory implements AbstractParserFactory {

    @Override
    public XMLParser getParserInstance(String parserType) {

        switch(parserType){
            case "NYERROR": return new NYErrorXMLParser();
            case "NYFEEDBACK": return new NYFeedbackXMLParser ();
            case "NYORDER": return new NYOrderXMLParser();
            case "NYRESPONSE": return new NYResponseXMLParser();
        }

        return null;
    }

}
```

```
package com.javacodegeeks.patterns.abstractfactorypattern;

public class TWParserFactory implements AbstractParserFactory {

    @Override
    public XMLParser getParserInstance(String parserType) {

        switch(parserType){
            case "TWERROR": return new TWErrorXMLParser();
            case "TWFEEDBACK": return new TWFeedbackXMLParser ();
            case "TWORDER": return new TWOrderXMLParser();
            case "TWRESPONSE": return new TWResponseXMLParser();
        }

        return null;
    }

}
```

The above two factories implement the `AbstractParserFactory` interface and overrides the `getParserInstance` method. It returns the client specific parser object, according to the parser type requested in the argument.

```
package com.javacodegeeks.patterns.abstractfactorypattern;

public interface XMLParser {

    public String parse();

}
```

The above interface is implemented by the concrete parser classes to parse the XMLs and returns the string message.

There are two clients and four different type of messages exchange between the company and its client. So, there should be six different types of concrete XML parsers that are specific to the client.

```
package com.javacodegeeks.patterns.abstractfactorypattern;

public class NYErrorXMLParser implements XMLParser{

    @Override
    public String parse() {
        System.out.println("NY Parsing error XML...");
    }

}
```

```
        return "NY Error XML Message";
    }
}
```

```
package com.javacodegeeks.patterns.abstractfactorypattern;

public class NYFeedbackXMLParser implements XMLParser{

    @Override
    public String parse() {
        System.out.println("NY Parsing feedback XML...");
        return "NY Feedback XML Message";
    }
}
```

```
package com.javacodegeeks.patterns.abstractfactorypattern;

public class NYOrderXMLParser implements XMLParser{

    @Override
    public String parse() {
        System.out.println("NY Parsing order XML...");
        return "NY Order XML Message";
    }
}
```

```
package com.javacodegeeks.patterns.abstractfactorypattern;

public class NYResponseXMLParser implements XMLParser{

    @Override
    public String parse() {
        System.out.println("NY Parsing response XML...");
        return "NY Response XML Message";
    }
}
```

```
package com.javacodegeeks.patterns.abstractfactorypattern;

public class TWErrorXMLParser implements XMLParser{

    @Override
    public String parse() {
        System.out.println("TW Parsing error XML...");
        return "TW Error XML Message";
    }
}
```

```
package com.javacodegeeks.patterns.abstractfactorypattern;

public class TWFeedbackXMLParser implements XMLParser{

    @Override
    public String parse() {
        System.out.println("TW Parsing feedback XML...");
    }
}
```

```
        return "TW Feedback XML Message";
    }
}
```

```
package com.javacodegeeks.patterns.abstractfactorypattern;

public class TWOrderXMLParser implements XMLParser{

    @Override
    public String parse() {
        System.out.println("TW Parsing order XML...");
        return "TW Order XML Message";
    }
}
```

```
package com.javacodegeeks.patterns.abstractfactorypattern;

public class TWResponseXMLParser implements XMLParser{

    @Override
    public String parse() {
        System.out.println("TW Parsing response XML...");
        return "TW Response XML Message";
    }
}
```

To avoid a dependency between the client code and the factories, optionally we implemented a factory-producer which has a static method and is responsible to provide a desired factory object to the client object.

```
package com.javacodegeeks.patterns.abstractfactorypattern;

public final class ParserFactoryProducer {

    private ParserFactoryProducer() {
        throw new AssertionError();
    }

    public static AbstractParserFactory getFactory(String factoryType) {

        switch(factoryType)
        {
            case "NYFactory": return new NYParserFactory();
            case "TWFactory": return new TWParserFactory();
        }

        return null;
    }
}
```

Now, let's test the code.

```
package com.javacodegeeks.patterns.abstractfactorypattern;

public class TestAbstractFactoryPattern {

    public static void main(String[] args) {
```

```

        AbstractParserFactory parserFactory = ParserFactoryProducer.getFactory(" ←
            NYFactory");
        XMLParser parser = parserFactory.getParserInstance("NYORDER");
        String msg="";
        msg = parser.parse();
        System.out.println(msg);

        System.out.println("*****");

        parserFactory = ParserFactoryProducer.getFactory("TWFactory");
        parser = parserFactory.getParserInstance("TWFEEDBACK");
        msg = parser.parse();
        System.out.println(msg);
    }
}

```

The above code will result to the following output:

```

NY Parsing order XML...
NY Order XML Message
*****
TW Parsing feedback XML...
TW Feedback XML Message

```

In the above class, we first got the NY factory from the factory producer, and then the Order XML parser from the NY parser factory. Then, we called the `parse` method on the parser object and displayed the return message. We did same for the TW client as clearly shown in the output.

14.4 When to use the Abstract Factory Design Pattern

Use the Abstract Factory pattern when

- A system should be independent of how its products are created, composed, and represented.
- A system should be configured with one of multiple families of products.
- A family of related product objects is designed to be used together, and you need to enforce this constraint.
- You want to provide a class library of products, and you want to reveal just their interfaces, not their implementations.

14.5 Abstract Factory Pattern in JDK

- `java.util.Calendar#getInstance()`
- `java.util.Arrays#asList()`
- `java.util.ResourceBundle#getBundle()`
- `java.sql.DriverManager#getConnection()`
- `java.sql.Connection#createStatement()`
- `java.sql.Statement#executeQuery()`
- `java.text.NumberFormat#getInstance()`
- `javax.xml.transform.TransformerFactory#newInstance()`

14.6 Download the Source Code

This was a lesson on the Abstract Factory Design Pattern. You may download the source code here: [AbstractFactoryPattern-Project](#)
